

ASSIGNMENT 3 REPORT - RAW SOCKET PACKET ANALYSIS AND PROTOCOL INVESTIGATION

1. OBJECTIVE

The objective of this assignment is to capture and inspect raw IP packets using Linux raw sockets, extract mandatory IP header fields, extract protocol-specific TCP fields, and validate the observations against Wireshark capture evidence.

2. STUDENT AND SYSTEM CONFIGURATION

- * Name: Sonkalpa Borah
- * Roll No: CS-BTC24-08
- * Last digit: '8'
- * Assigned protocol: 'TCP' ('ip.proto == 6')
- * Environment: Ubuntu on WSL2

System details (from 'system_details.txt'):

- * 'uname -a': Linux 5.15.167.4-microsoft-standard-WSL2 x86_64
- * 'gcc --version': GCC 13.3.0 (Ubuntu 24.04)
- * capture interface used: 'lo'

3. COMPILE AND RUN COMMANDS

- * 'gcc raw_capture.c -o raw_capture'
- * 'sudo ./raw_capture 20'

Traffic generation command used:

- * 'python3 generate_tcp_traffic.py --count 150 --port 5000 --delay 0.02 --startup-delay 0.8'

Capture command used:

- * 'sudo dumpcap -i lo -a duration:20 -w capture.pcapng'

4. PROGRAM DESIGN

'raw_capture.c' uses 'socket(AF_INET, SOCK_RAW, IPPROTO_TCP)' for TCP packet capture and prints details for 20 packets.

Per packet, the program prints:

- * 'SRC_IP', 'DST_IP'

- * 'PROTOCOL', 'PROTOCOL_NO'
- * 'TTL'
- * 'PACKET_SIZE'

TCP-specific fields:

- * 'TCP_SRC_PORT'
- * 'TCP_DST_PORT'
- * 'TCP_FLAGS' (decoded as 'SYN', 'ACK', 'FIN', 'PSH', etc.)

Program enhancement field (Task 5):

- * 'IP_IDENTIFICATION'

5. TRAFFIC GENERATION

Traffic was generated locally on loopback by repeatedly creating TCP connections between a client and local server ('127.0.0.1:5000').

- * command: 'python3 generate_tcp_traffic.py --count 150 --port 5000 --delay 0.02 --startup-delay 0.8'
- * interface observed: 'lo'
- * output file: 'traffic_generation_output.txt'

Observed output:

- * 'Generated 150 TCP connections to 127.0.0.1:5000'

6. EXPERIMENTAL RESULTS

The program captured 20 TCP packets successfully ('program_output.txt') with the required format:

- * 'ROLL_NO=CS-BTC24-08'
- * 'ASSIGNED_PROTOCOL=TCP'
- * 'PACKET_NO=1..20'

Example first packet:

- * 'SRC_IP=127.0.0.1'
- * 'DST_IP=127.0.0.1'
- * 'PROTOCOL_NO=6'
- * 'TTL=64'
- * 'PACKET_SIZE=60'
- * 'TCP_SRC_PORT=37286', 'TCP_DST_PORT=5000', 'TCP_FLAGS=SYN'

7. WIRESHARK VERIFICATION

Capture file: 'capture.pcapng'

Display filter used: 'tcp' (equivalent to 'ip.proto == 6').

Five-packet comparison table (from 'packet_comparison_template.csv'):

Wireshark Frame	Source IP	Destination IP	Protocol	TTL	Packet Size	Program Output Matched
-----------------	-----------	----------------	----------	-----	-------------	------------------------

---	:	---	---	---	---	---
-----	---	-----	-----	-----	-----	-----

1		127.0.0.1		127.0.0.1		TCP		64		60		Yes
---	--	-----------	--	-----------	--	-----	--	----	--	----	--	-----

2		127.0.0.1		127.0.0.1		TCP		64		60		Yes
---	--	-----------	--	-----------	--	-----	--	----	--	----	--	-----

3		127.0.0.1		127.0.0.1		TCP		64		52		Yes
---	--	-----------	--	-----------	--	-----	--	----	--	----	--	-----

4		127.0.0.1		127.0.0.1		TCP		64		60		Yes
---	--	-----------	--	-----------	--	-----	--	----	--	----	--	-----

5		127.0.0.1		127.0.0.1		TCP		64		52		Yes
---	--	-----------	--	-----------	--	-----	--	----	--	----	--	-----

8. HEADER ANALYSIS (3 PACKETS)

Packets analyzed: Frame 1, Frame 4, Frame 6.

1. Why is TTL not zero?

- TTL is initialized by sender (64 here) and decremented per router hop. On loopback ('127.0.0.1'), packets do not traverse routers, so TTL remains 64 and never reaches zero.

2. Why do packet sizes differ?

- Different TCP segments carry different payload/control content. For example, SYN packets are 60 bytes, pure ACK packets are 52 bytes, and packets with payload ('PSH,ACK') can be 54 or 60 bytes.

3. What information does protocol field provide?

- It identifies the transport protocol carried by IP. Here 'PROTOCOL_NO=6' means TCP.

4. What could happen if protocol field is modified?

- The receiver/network stack may pass the packet to the wrong protocol handler, causing parsing failure or packet drop.

5. What do protocol-specific fields reveal?

- TCP source/destination ports identify flow endpoints, and TCP flags reveal connection phase (SYN handshake, ACK confirmation, PSH data, FIN close).

9. PROGRAM ENHANCEMENT

Selected additional IP field: 'IP_IDENTIFICATION'.

Purpose:

- * Used by IP for fragmentation and reassembly; fragments of the same original datagram share the same identification value.

Observed values from five packets:

- * Packet 1: '43682'
- * Packet 2: '0'
- * Packet 3: '43683'
- * Packet 4: '43684'
- * Packet 5: '24180'

Observation:

- * The value changes across packets (not constant), reflecting different packets generated by the stack. One packet shows '0', which is also valid for certain local traffic behavior.

10. REFLECTION ANSWERS (<= 300 WORDS)

Raw sockets require root privileges because they allow direct access to packet headers and low-level traffic that can bypass normal transport-layer safety controls. This access can be abused for spoofing, scanning, or packet manipulation, so the OS restricts it.

Raw sockets differ from normal TCP/UDP sockets because applications receive and interpret headers directly. In TCP/UDP sockets, the kernel handles most protocol logic (connection state, reliability, segmentation/reassembly, checksums) and applications read only payload-oriented streams/datagrams.

One advantage of raw sockets is full visibility/control of packet-level behavior, useful for protocol debugging and security tooling. One limitation is higher complexity and privilege requirements; implementation mistakes can easily cause incorrect parsing or unstable behavior.

One cybersecurity application is intrusion detection or custom packet inspection where tools analyze live header/flag patterns (for example, SYN flood patterns or anomalous protocol usage).

11. CONCLUSION

The TCP raw socket implementation successfully captured and decoded 20 packets with required IP and TCP fields. Generated traffic was verified against

Wireshark/dumpcap capture, and all selected comparison rows matched for source, destination, protocol, TTL, and packet size.